Josephus problem simulation with linked list

> Help me design a Java program intended to simulate the Josephus problem. To start the simulation, the program will get the names of the people in the circle from a text file. Then, it will prompt the user to input the elimination count. For each run of the simulation, the name of the person eliminated will be printed. Following that, the names of survivors and the count will be printed. This simulation will run until one survivor remains. At the end, the program will verify its result with a mathematical solution to find out who survives. The program should be designed with an object oriented approach, and it must make use of a linked list.

Check frontend design skill

Here's a full object-oriented Java design for the Josephus problem simulation. I'll create all the necessary files:

Created 6 files

Now let's compile and run it to verify correctness:

Ran 4 commands

No compiler available in this sandbox, but the code is complete and ready. Let me copy all files to the output directory:

Ran a command, read a file

Here's a breakdown of the full design:

Program Architecture

The project is split into **5 classes**, each with a single, focused responsibility:

`Person.java` — The Node

A simple data class representing one person in the circle. Holds a `name` and a `next` pointer — the building block of the linked list. Encapsulates its fields with proper getters/setters.

`JosephusCircle.java` — The Circular Linked List

The heart of the simulation. Maintains a **circular singly linked list** with `head` and `tail` pointers. Key design decisions:

- `add(name)` appends to the tail and rewires `tail.next → head` to keep it circular.
- `eliminate(k)` walks `k-1` steps to find the node *just before* the victim, splices it out, then advances `head` to start the next count right after the gap. This correctly handles removing the head or tail node.
- `listRemaining()` and `getSize()` support reporting after each round.

`JosephusMath.java` — The Verifier

Implements the classic **iterative recurrence**:

```
pos = 0
for i from 2 to n:  pos = (pos + k) % i
```

This runs in O(n) time and produces the 0-based survivor index, which is mapped back to the original name list for comparison.

`NamesFileReader.java` — File I/O Utility

Reads names from a text file (one per line), skipping blank lines and `#` comment lines. Encapsulates all file-handling logic separately from simulation logic.

`JosephusSimulation.java` — The Driver

Orchestrates everything: loads the file, prompts for `k`, builds the circle, runs the elimination loop printing each round, then calls `JosephusMath` to verify the result.

How to Run

bash

```
# Compile all files
javac *.java

# Run (file path optional — defaults to names.txt)
java JosephusSimulation
```

```
# or pass the file explicitly:
java JosephusSimulation names.txt
```

You'll be prompted for `k`, then the simulation runs round by round. Edit `names.txt` to add your own participants (one name per line).

Josephussimulation Code · JAVA	Download
Josephuscircle Code · JAVA	Download
Person Code · JAVA	Download
Josephusmath Code · JAVA	Download
Namesfilereader Code · JAVA	Download
Names TXT	Download

Download all

Let's now add a UI, so the Josephus simulation can be visualized. With the same specifications, use a visual circle with names around the perimeter. An arrow from the center would point at each name it iterates through, while a counter will keep track of how far the arrow has iterated. To eliminate a name, remove it from the circle, and add it to the list on the right hand side of the screen.

Josephus problem

Enter names (one per line), set the elimination count, then start.

```
Bob
Charlie
Diana
Edward
Fiona
George
Hannah
Ivan
Julia
```

Elimination count k 3 Start simulation

Here's how the simulation works:

The circle of names is drawn around the perimeter, each person as a labeled node. A golden arrow sweeps out from the center and steps through each person one at a time — the counter in the middle tracks how far into the current count you are. When it reaches k, the highlighted node fades out and slides into the **Eliminated** list on the right, numbered in order. The circle then redraws with one fewer person, and counting resumes from the next position.

Controls:

- **Step** — advance one full elimination at a time, watching the arrow sweep and count
- **Auto play** / **Pause** — runs the simulation continuously
- **Speed slider** — ranges from slow (great for following along) to fast (for large circles)
- **Restart** — takes you back to the setup screen to change names or k

At the end, a single survivor remains in the circle, and the result is verified against the classic Josephus recurrence formula `J(n,k) = (J(n-1,k) + k) % n` — confirming the simulation is correct.

I want Java files that I can add to my project

Check existing Java files >

The UI from the previous step was built entirely in JavaScript (it runs in the browser). To bring that same visualization into Java, I'll create a Swing-based GUI that mirrors it faithfully — circle of names, sweeping arrow, counter, eliminated list on the right.

Viewed 2 files >

Now I'll create the GUI files. The design uses a `CirclePanel` for custom rendering and a `SimulationController` to drive the animation via a `javax.swing.Timer`.

Created 4 files, ran a command, read a file >

Four new Java files to drop straight into your project alongside the originals. Here's what each one does:

New files:

`SimulationState.java` — a plain data class shared between the GUI and the controller. It holds the alive list, the current arrow index, step count, eliminating node index/alpha, and the eliminated log. Neither the panel nor the controller owns the state — they both read and write through this object.

`CirclePanel.java` — a custom `JPanel` that overrides `paintComponent` using Java2D. It draws the ring, each name as a labelled circle, the golden arrow with a dashed trail and filled arrowhead, and the hub counter in the centre. When the simulation ends it overlays a banner with the survivor's name and the mathematical verification result.

`SimulationController.java` — drives all animation via a `javax.swing.Timer` (EDT-safe). Each tick is either a *count step* (arrow advances one position, counter increments) or a *fade step* (eliminating node's alpha decreases). After the fade completes it splices the node from the alive list and fires the `onElimination` callback so the GUI can update the side list.

`JosephusGUI.java` — the main window. Uses a `CardLayout` to swap between a **setup screen** (names text area, k spinner, load-from-file button) and a **simulation screen** (circle panel + eliminated list + Step / Auto play / Restart / Speed controls). Launch it with:

```bash
javac *.java
java JosephusGUI
```

The original five files (`Person`, `JosephusCircle`, `JosephusMath`, `NamesFileReader`, `JosephusSimulation`) are unchanged — `JosephusSimulation` still works as the terminal entry point if you prefer the console version.

Josephusgui
Code · JAVA
Download

Circlepanel
Code · JAVA
Download

Simulationcontroller
Code · JAVA
Download

Simulationstate
Code · JAVA
Download

Person
Code · JAVA
Download

Josephuscircle
Code · JAVA
Download

Josephusmath
Code · JAVA
Download

Namesfilereader
Code · JAVA
Download

Download all